

YELLOWBE
РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Содержание

Сокращения и определения	3
1. Общие сведения	3
1.1. Общее описание.....	3
1.2. Компоненты дизайн-системы.....	3
2. Начало работы.....	4
2.1. Дизайн-токены	4
2.1.1. CSS	4
2.1.2 SCSS	5
2.1.3 JS.....	6
2.1.4 Assets.....	7
2.1.5 Подключение favicon.....	8
2.1.6 Вспомогательные файлы.....	8
2.2 Компоненты дизайн системы	9
2.2.1 Глобальные настройки	9
2.2.2 Шрифты и вспомогательные сущности.....	9
2.2.3 Использование компонентов из библиотеки.....	10
2.3 Иконочный шрифт.....	10
2.3.1 Лигатуры.....	11
2.3.2 Псевдо-элементы	12
2.3.3 Стилизация	12
3. Дизайн.....	12
3.1. Работа с компонентами.....	12
3.1.2 Библиотека в Pixso.....	13
3.1.3 Как найти компонент.....	13

Сокращения и определения

Брендбук — это основная философия бренда. Свободно работаем с визуальным языком в коммуникациях, рекламных кампаниях и спецпроектах.

Дизайн-токены – это значения стиля элементов интерфейса, таких как цвет, типографика, интервалы, тени и т.д. Токены используются вместо хардкодных значений, чтобы обеспечить единство интерфейсов и сохранить свободу выбора технологий для конечных продуктов.

1. Общие сведения

1.1. Общее описание

Дизайн-платформа представляет из себя набор компонентов, правил, предписаний и инструментов для повышения качества и скорости разработки продуктов, а также эффективной поддержки существующих.

Yellowbe упрощает и ускоряет процесс разработки интерфейсов за счёт токенов, готовых компонентов, инструментов и единых принципов.

1.2. Компоненты дизайн-системы

Библиотека компонентов Pixso;

Набор ассетов: логотипы, иконки, продуктовые иконки, логотипы продуктов, шрифты и др.;

Руководство Handbook;

Тачпоинты для дизайна/разработки;

Дизайн-токены;

Storybook с компонентами React;

Библиотека компонентов React;

Библиотека токенов.

2. Начало работы

2.1. Дизайн-токены

Дизайн-токены существуют в трех форматах:

- CSS
- SCSS
- JS

Для корректной работы токенов компонентов обязательно необходимо подключить к проекту токены темы. Информацию по подключению тем см. в гайде Theming.

2.1.1. CSS

Токены в формате css-переменных.

Необходимо подключить требуемые файлы в проект (файлы темы, файл с переменными компонентов) и в кастомных стилях вместо конечных значений использовать css-переменные.

Пример:

```
.button {  
  
    background-color: var(--button-background-color);  
  
    padding: var(--size-spacing-x-2);  
  
}
```

Структура:

node_modules/@beeline/design-tokens/css

```
├── font-face.css  
├── iconfont  
|   ├── iconfont.css  
|   └── icons.css  
|  
└── tokens  
    ├── globals  
    |   ├── index.css  
    |   ├── elevation.css  
    |   ├── colors.css  
    |   ├── fonts.css  
    |   ├── palettes.css  
    |   └── sizes.css  
    └── themes
```

```

├──dark.css
└──light.css

```

2.1.2 SCSS

Токены в формате scss-переменных.

Переменные разбиты по категориям и компонентам, поэтому можно подключать только те файлы, которые необходимы. Далее, при стилизации, в кастомных стилях нужно использовать данные scss-переменные.

Токены рекомендуется подключать через SCSS Modules, с использованием синтаксиса `@use`.

Пример:

```

@use '~@beeline/design-tokens/scss/tokens/components/button';

.button {

  background-color: button.$button-background-color;

  padding: button.$button-padding-horizontal;

}

```

Структура:

```

node_modules/@beeline/design-tokens/scss

├──font-face.scss
├──iconfont
│   ├──iconfont.scss
│   └──icons.scss
│
└──tokens

├──globals
│
│   ├── palettes.scss // палитры базовых цветов, НЕ рекомендованы для использования в
проектах - см.
    страницу Темы
    ├──_colors.scss
    ├──_elevation.scss
    ├──_fonts.scss
    ├──_sizes.scss
    └──index.scss
└──themes

```

```

| |—_index.scss
| |—_dark.scss
| |—_light.scss
| |—_theme.scss
| |—_theme-variables.scss
|—components

```

2.1.3 JS

Токены в формате ES6-переменных служат для использования в рамках подхода CSS-in-JS в целом (и с библиотекой Styled Components в частности).

Переменные разбиты по категориям и компонентам, поэтому можно подключать только те файлы, которые необходимы.

Все JavaScript файлы сопровождаются файлами .d.ts типизации для Typescript.

Глобальные токены (цвета, размеры и т.д.) можно подключать двумя способами:

1. Импортируя конкретные переменные из пакета токенов

```

import { colorTextLinkVisited } from "@beeline/design-tokens/js/tokens";

const someValue = colorTextLinkVisited;

```

2. Импортируя соответствующий модуль, содержащий группу значений

```

import { color } from "@beeline/design-tokens/js/tokens";

const someOtherValue = color.colorTextLinkVisited;

```

Токены компонентов рекомендуется импортировать только через модуль:

```

import { button } from "@beeline/design-tokens/js/tokens";

const anotherValue = button.buttonBorderColorFocus;

```

Структура:

```

node_modules/@beeline/design-tokens/js
|—iconfont
| |—index.js // экспорт enum с иконками
| |—icons.js // enum со всеми иконками
|
|—tokens
|—index.js // экспорт глобальных токенов и токенов компонентов
|—globals
| |—colors.js

```

```

| |—elevation.js
| |—fonts.js
| |—sizes.js
|—themes
| |—index.js // экспорт тем
| |—dark.js
| |—light.js
|—components
|—avatar.js
|—...
|—tooltip.js

```

2.1.4 Assets

Ассеты включают: шрифты, иконки, логотип, favicon. На этапе сборки, в зависимости от особенностей вашего сборщика, данные assets необходимо включить в сборку.

```

node_modules/@beeline/design-tokens/assets
|—favicon
| |—favicon.ico // "классическая" favicon
| |—icon.svg //иконка в svg (современные браузеры возьмут ее)
| |—apple-touch-icon.png // иконка для добавления сайта на домашний экран
устройств от Apple
| |—icon-192.png // иконка для использование в web app manifest для Android
| |—icon-512.png // иконка для использование в web app manifest для Android
|—logo
| |—logo-light-theme.svg // лого с темным текстом
| |—logo-dark-theme.svg // лого со светлым текстом
| |—logo-themeable.svg // лого, в котором цвет текста завязан на CSS Variable
(подробнее см. гайд
"Theming")
|—fonts
| |—iconfont
| | |—BeelineIcons.ttf
| | |—BeelineIcons.woff
| | |—BeelineIcons.woff2
| |
| |—beeline-sans

```

```

| |——BeelineSans-Black.ttf
| |——BeelineSans-Black.woff
| |——BeelineSans-Black.woff2
|
|——icons
|——add.svg
|——alarm.svg
|——...

```

2.1.5 Подключение favicon

Для покрытия большинства потребностей в задании favicon достаточно 5 файлов, представленных в assets дизайн-платформы.

```

<link rel="icon" href="/favicon.ico" sizes="any"><!-- 32x32 -->

<link rel="icon" href="/icon.svg" type="image/svg+xml">

<link rel="apple-touch-icon" href="/apple-touch-icon.png"><!-- 180x180 -->

<link rel="manifest" href="/manifest.webmanifest">

// manifest.webmanifest

{
  "icons": [
    { "src": "/icon-192.png", "type": "image/png", "sizes": "192x192" },
    { "src": "/icon-512.png", "type": "image/png", "sizes": "512x512" }
  ]
}

```

2.1.6 Вспомогательные файлы

Вспомогательные файлы и сущности, в том числе для быстрого подключения assets.

Делятся на 2 типа:

1. файл настроек
2. файл с вспомогательными функциями

Файл настроек

Содержит в себе готовый код для подключения функционала, такой файл нужно только импортировать в глобальном файле стилей

- **font-face.{scss, css}**

Содержит @font-face для подключения всех начертаний шрифтов, предоставляемых дизайн-платформой.

Для scss можно настроить путь до шрифтов, переопределив переменную(ые) `$font-path-{название шрифта}`.

- **iconfont.{scss, css}**

Содержит `@font-face` и стили класса `beeline-icons` для подключения иконочного шрифта, предоставляемого дизайн-платформой.

Для scss можно настроить путь до шрифтов, переопределив переменную `$font-path-iconfont`.

Подробнее см. гайд "Icon font"

- **scrollbar.{scss, css}**

Требует подключения токенов темы. Смотри раздел темизация

Переопределяет стили браузерного сколлбара для соответствия ДС

По-умолчанию поддерживает темизацию

Работает только в chromium-браузерах

Файл с вспомогательными функциями

Содержит в себе функции, которые можно импортировать и использовать на любом уровне

- **Миксины SCSS**

Миксины для быстрой стилизации, например - для типографики.

2.2 Компоненты дизайн системы

2.2.1 Глобальные настройки

Если в приложении должна быть подключена темизация, есть 3 пути:

если у вас уже настроено использование токенов темы из пакета `@beeline/design-tokens`
- ничего больше делать не нужно

подключить темизацию по инструкции из пакета `@beeline/design-tokens`

Обернуть приложение в компонент `ThemeProvider` из библиотеки

```
<ThemeProvider isRoot theme='light'>  
  <App />  
</ThemeProvider>
```

2.2.2 Шрифты и вспомогательные сущности

По аналогии с иконочным шрифтом можно подключить и обычный, а также другие полезные сущности, например перестилизацию браузерного скроллбара, поддерживающую темизацию.

Для этого подключите соответствующие css или scss файлы где угодно на глобальном уровне

Например можно добавить такой код в `index.{css,scss}`

```
@import '~@beeline/design-tokens/css/iconfont/iconfont.css';  
@import '~@beeline/design-tokens/css/font-face.css';  
@import '~@beeline/design-tokens/css/scrollbar.css';  
или такой в index.{js,ts,jsx,tsx}
```

```
import '~@beeline/design-tokens/css/iconfont/iconfont.css';
import '~@beeline/design-tokens/css/font-face.css';
import '~@beeline/design-tokens/css/scrollbar.css';
```

После этого в вашем приложении:

- станет доступен шрифт "Roboto" (с ним по умолчанию работает компонент Typography)
- появится иконочный шрифт "Beeline Icons" (используется компонентом Icon)
- изменится вид браузерного скроллбара (работает в Chromium и Firefox)

ВНИМАНИЕ: подключение шрифтов таким образом зависит от ваших настроек сборщика. Чаще всего сборка реакт настроена таким образом, что это сработает корректно. Если вы сталкиваетесь с проблемами:

- обратитесь к более низкоуровневым описаниям подключения ассетов и иконочного шрифта
- пишите в наш чат, будем рады помочь

2.2.3 Использование компонентов из библиотеки

Библиотека поддерживает стандарты CommonJS и ESM, для использования ее компонентов можно использовать и require и import синтаксис.

Пример использование ESM

```
import { Button, Typography } from '@beeline/design-system-react';
```

```
<Button>Click</Button>
<Typography>Text</Typography>
```

Пример использование CommonJS

```
const { Button, Typography } = require('@beeline/design-system-react');
```

```
<Button>Click</Button>
<Typography>Text</Typography>
```

2.3 Иконочный шрифт

Иконочный шрифт создан для удобной работы с векторными иконками в веб-приложениях.

Структура файлов:

```
node_modules/@beeline/design-tokens
├── assets
│   ├── fonts
│   ├── iconfont
│   └── BeelineIcons.woff
```

```

| |——BeelineIcons.woff
| |——BeelineIcons.woff2
|——scss
| |——iconfont
| |——iconfont.scss
| |——icons.scss
|
|——css
| |——iconfont
| |——iconfont.css
| |——icons.css
|
|——js
|——iconfont
|——icons.js
|——index.js

```

Для подключения шрифта необходимо:

1. включить файлы шрифта в сборку проекта
2. подключить файл с `@font-face` и базовыми стилями(`iconfont.{scss, css}`) в проект (или задать данные стили вручную, если в npm-пакете нет подходящего формата файла с `@font-face`)

По умолчанию шрифты ожидаются по адресу `assets/fonts/iconfont/` файлы шрифта. Для scss можно настроить путь до шрифтов, переопределив переменную `$font-path-iconfont`.

Существуют два способа добавления иконок в HTML-страницу: с помощью лигатуры и через псевдо-элементы

2.3.1 Лигатуры

Для использования лигатур необходимо элементу, который будет содержать иконку, дать класс `beeline-icons` и указать название требуемой иконки между открывающим и закрывающим тегами.

Обычно, для добавления иконок используется элемент `<i></i>`, но могут использоваться и другие.

Пример:

```
<i class="beeline-icons">alarm</i>
```

Для проектов, которые используют Typescript, также есть строковый enum Icons, содержащий перечисление всех иконок, для удобства использования в коде.

Импорт enum:

```
import { Icons } from '@beeline/design-tokens/js/iconfont'
```

2.3.2 Псевдо-элементы

Чтобы добавить иконку в качестве псевдо-элемента `::before`, необходимо элементу, который будет содержать иконку, дать класс `beeline-icons` и класс иконки. Класс иконки состоит из префикса `beeline-icons` и имени иконки.

Пример:

```
<i class="beeline-icons beeline-icons-add"></i>
```

2.3.3 Стилизация

Изменение размера и цвета иконок при использовании иконочного шрифта осуществляется стандартными свойствами CSS для работы со шрифтами: `color` и `font-size`.

Пример:

```
.close-icon {  
  color: red;  
  
  font-size: 18px;  
}
```

3. Дизайн

3.1. Работа с компонентами

Компоненты — это часто повторяющиеся элементы интерфейса. Они помогают создавать и поддерживать консистентный дизайн в продуктах. Мы пытаемся покрывать все потребности дизайнеров и постоянно работаем над развитием системы.

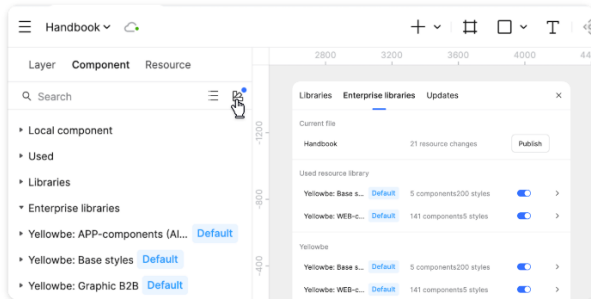
3.1.1 Принципы работы с компонентами

При использовании библиотеки компонентов следуйте принципам:

1. Выбирать компоненты через вкладку Component
2. Не отвязывать компоненты, с целью их “доработать”
3. Не копировать мастер-компонент из библиотеки в Pixso
4. Если нет нужного компонента или иконки — связаться с [командой Дизайн-платформы](#) с запросом на добавление
5. При обнаружении некорректной работы компонента — описать суть проблемы через форму багов

3.1.2 Библиотека в Pixso

Если вы впервые используете Pixso, проверьте что у вас включен набор библиотек Yellowbe. Это позволит видеть компоненты на вкладке Component и панель стилей справа.



3.1.3 Как найти компонент

Прямой путь к актуальным компонентам через вкладку Component.

1. Раскройте вкладку Enterprise libraries
2. Выберите любую библиотеку, например, Yellowbe: WEB-components
3. Нажмите на раздел с именем компонента
4. Перетащите компонент в рабочую область

